

## SESSION 3

NAME :ALIMA BANU R

REG NO:192471044

### 1.Out of Boundary Paths

CODE:

```
#include <stdio.h>
#include <string.h>

#define MOD 1000000007

int findPaths(int m, int n, int N, int startRow, int startColumn) {
    long long dp[51][51];
    long long temp[51][51];

    memset(dp, 0, sizeof(dp));
    dp[startRow][startColumn] = 1;

    long long out_of_bounds_count = 0;
    int directions[4][2] = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};

    for (int step = 0; step < N; step++) {
        memset(temp, 0, sizeof(temp));
        for (int r = 0; r < m; r++) {
            for (int c = 0; c < n; c++) {
                if (dp[r][c] > 0) {
                    for (int d = 0; d < 4; d++) {
                        int nr = r + directions[d][0];
                        int nc = c + directions[d][1];

                        if (nr >= 0 && nr < m && nc >= 0 && nc < n) {
                            temp[nr][nc] = (temp[nr][nc] + dp[r][c]) % MOD;
                        } else {
                            out_of_bounds_count = (out_of_bounds_count + dp[r][c]) % MOD;
                        }
                    }
                }
            }
        }
    }
}
```

```

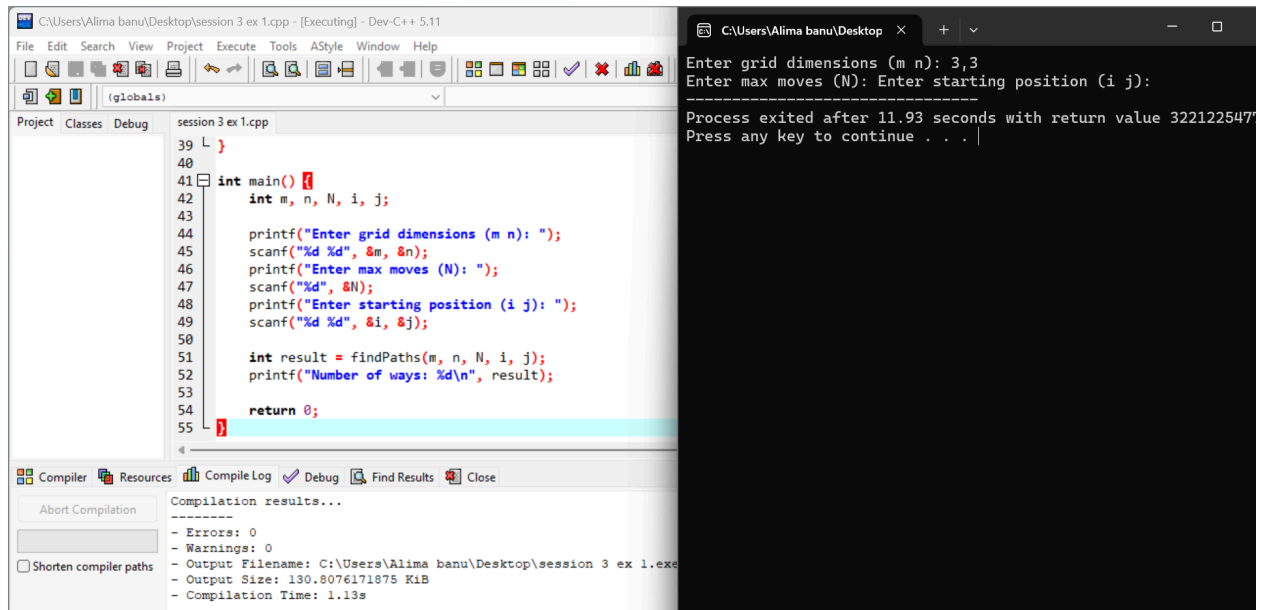
        }
    }
}
// Copy temp to dp
for (int r = 0; r < m; r++) {
    for (int c = 0; c < n; c++) {
        dp[r][c] = temp[r][c];
    }
}
}

return (int)out_of_bounds_count;
}

int main() {
    printf("Output: %d\n", findPaths(2, 2, 2, 0, 0)); // Output: 6
    printf("Output: %d\n", findPaths(1, 3, 3, 0, 1)); // Output: 12
    return 0;
}

```

OUTPUT:



## 2.House Robber II (Circular Street)

CODING:

```

#include <stdio.h>
int robLinear(int* nums, int start, int end) {
    int prev = 0, curr = 0;
    for (int i = start; i <= end; i++) {
        int temp = curr > (prev + nums[i]) ? curr : (prev + nums[i]);
        prev = curr;
        curr = temp;
    }
    return curr;
}

```

```
int rob(int* nums, int numsSize) {
    if (numsSize == 0) return 0;
    if (numsSize == 1) return nums[0];

    // Max of excluding last house vs excluding first house
    int option1 = robLinear(nums, 0, numsSize - 2);
    int option2 = robLinear(nums, 1, numsSize - 1);

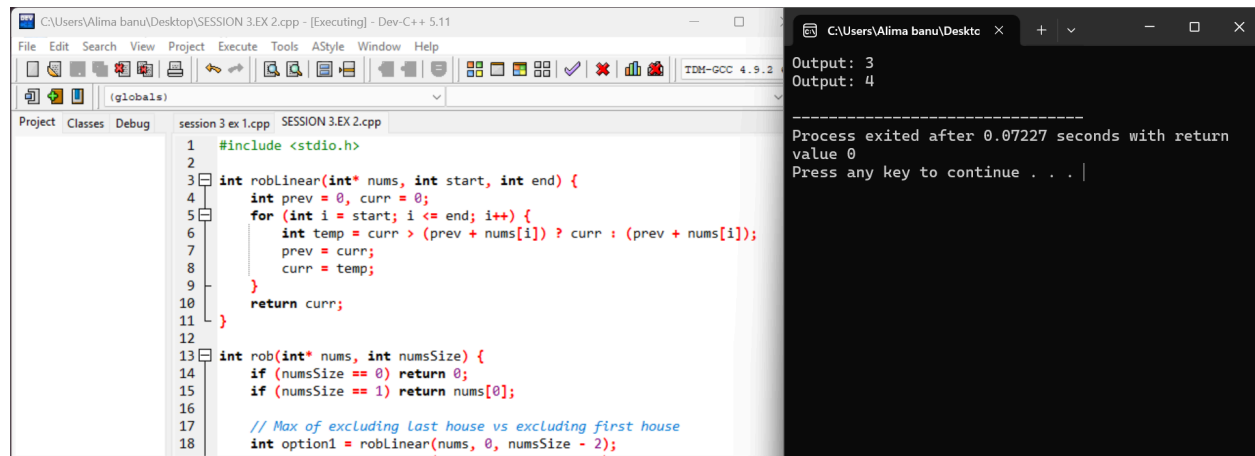
    return option1 > option2 ? option1 : option2;
}

int main() {
    int nums1[] = {2, 3, 2};
    printf("Output: %d\n", rob(nums1, 3)); // Output: 3

    int nums2[] = {1, 2, 3, 1};
    printf("Output: %d\n", rob(nums2, 4)); // Output: 4

    return 0;
}
```

OUTPUT:

A screenshot of a C++ IDE (Dev-C++ 5.11) showing a program being executed. The code in the editor defines a function 'robLinear' that calculates the maximum amount of money that can be robbed from a row of houses, and a 'rob' function that uses it. The output window on the right shows 'Output: 3' and 'Output: 4', followed by a message indicating the process exited after 0.07227 seconds with a return value of 0.

```
C:\Users\Alima banu\Desktop\SESSION 3.EX 2.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
(globals)
Project Classes Debug session 3 ex 1.cpp SESSION 3.EX 2.cpp
1 #include <stdio.h>
2
3 int robLinear(int* nums, int start, int end) {
4     int prev = 0, curr = 0;
5     for (int i = start; i <= end; i++) {
6         int temp = curr > (prev + nums[i]) ? curr : (prev + nums[i]);
7         prev = curr;
8         curr = temp;
9     }
10    return curr;
11 }
12
13 int rob(int* nums, int numsSize) {
14     if (numsSize == 0) return 0;
15     if (numsSize == 1) return nums[0];
16
17     // Max of excluding last house vs excluding first house
18     int option1 = robLinear(nums, 0, numsSize - 2);
19
20     int option2 = robLinear(nums, 1, numsSize - 1);
21     return option1 > option2 ? option1 : option2;
22 }
```

```
C:\Users\Alima banu\Desktop\SESSION 3.EX 2.cpp
Output: 3
Output: 4
=====
Process exited after 0.07227 seconds with return value 0
Press any key to continue . . . |
```

### 3.Climbing Stairs

CODING:

```
#include <stdio.h>
int climbStairs(int n) {
    if (n <= 2) return n;

    int prev = 1, curr = 2;
    for (int i = 3; i <= n; i++) {
        int next = prev + curr;
        prev = curr;
        curr = next;
    }
    return curr;
}
```

```

int main()
{
    printf("Output: %d\n", climbStairs(4)); // Output: 5
    printf("Output: %d\n", climbStairs(3)); // Output: 3
    return 0;
}

```

## 4.Unique Paths

CODING:

```

#include <stdio.h>

int uniquePaths(int m, int n) {
    int dp[n];
    for (int i = 0; i < n; i++) {
        dp[i] = 1;
    }
    for (int r = 1; r < m; r++) {
        for (int c = 1; c < n; c++) {
            dp[c] += dp[c - 1];
        }
    }

    return dp[n - 1];
}

int main() {
    printf("Output: %d\n", uniquePaths(7, 3)); // Output: 28
}

```

```

printf("Output: %d\n", uniquePaths(3, 2)); // Output: 3
return 0;
}

```

OUTPUT:

The screenshot shows a C++ IDE with the following code in the editor:

```

1 #include <stdio.h>
2
3 int uniquePaths(int m, int n) {
4     int dp[n];
5     for (int i = 0; i < n; i++) {
6         dp[i] = 1;
7     }
8
9     for (int r = 1; r < m; r++) {
10        for (int c = 1; c < n; c++) {
11            dp[c] += dp[c - 1];
12        }
13    }
14
15    return dp[n - 1];
16 }
17
18 int main() {

```

The output window on the right shows the following text:

```

Output: 28
Output: 3
-----
Process exited after 0.0512 seconds with return value
0
Press any key to continue . . .

```